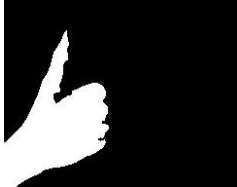


Team Geniuses SNN Application

Application Overview

For this competition, we have decided to create a model that classifies hand gestures. We used the following dataset [1]. This dataset contains 5,243 images of a hand, which is performing one of ten gestures, taken from OpenCV, and the numbers by the names are the labels given to each gesture type by the model. Given an image of a hand, the model should be able to classify which gesture it is showing.

Gesture Name	Example of Gesture	Gesture Name	Example of Gesture
Call Me (0)		Rock (5)	
Fingers Crossed (1)		Rock On (6)	
Okay (2)		Scissors (7)	
Paper (3)		Thumbs (8)	
Peace (4)		Up (9)	

One possible use of classifying hand gestures is the remote control of hardware, perhaps most excitingly robots and drones (of course the images themselves would need to be changed or added to). Robots and drones need to make small and precise movements, which is often difficult with a traditional remote controller. By recording and noting changes in hand movements, a finer degree of precision can be obtained.

Interpreting and translating sign language could be a second use of the recognition of gestures. A word, phrase or letter represented by a hand signal can be matched with its meaning in a written language, which can then be used to make a translation. However, an extremely large amount of data would be needed because of the number of different signs used.

SNN Architecture Details

One of the main advantages of using spiking neural networks (SNNs) is that they often have lower power consumption levels than a non-spiking network used to complete the same task [2]. This is because SNNs are much more closely based on the structure of a real brain. Instead of transmitting information at each propagation cycle, like a traditional artificial neural network (ANN), SNNs only transmit data when the value of a membrane potential, a property of the (LIF) neurons in an SNN, exceeds a threshold, which causes the neuron to fire and to generate a signal (spike) that affects the membrane potentials of other neurons. Hence, the computational effort is reduced as many neurons will not fire.

For our application, we have created a three layer convolutional SNN with 1,024 neurons in the hidden layer, where the first two layers (input and hidden) are made up of spiking convolutions (which are then pooled), and a fully connected output layer, which is also spiked. This structure was chosen because, among the different variations of ANN) convolutional neural networks (CNNs) have been observed to perform the best on image classification [2]. However, after applying a convolution to the images' data, rather than using an activation function (such as the ReLU function) on the convolution's output as in a traditional CNN, the output is converted to a current which can then cause a neuron to fire if this current produces a high enough membrane potential, creating a convolutional spiking layer. This process is known as converting an ANN to an SNN [3].

The images from the dataset have dimensions 240x195. As mentioned before, there are 5,243 images in the dataset, and these have been split into batches of size 8. The values of the pixels of the images are normalised between -1 and 1 to reduce bias. In order to process these more quickly, each image is resized to 28x28, as 28 is the highest multiple of 4 (the size of the tensor is halved after each spiking convolution) below 32, and any image larger than 32x32 cannot be used as otherwise the input layer would have over 1,024 neurons. 80% of the data is used for training, and the remaining 20% for testing, and these are randomly allocated to reduce the change of any gesture being underrepresented in either subset. The model is trained over ten epochs, in each epoch there are twenty training steps, and a learning rate of 0.0001 is used.

SNNTorch's Leaky [4] is used to generate the Leaky Integrate and Fire (LIF) neurons. We set the decay rate of the neurons' membrane potential (beta) to be 0.95.

In the first layer, a convolution with input channel size 1 (grayscale images) and output channel size 16 is applied to the input image. The result of the convolution gives a current, which is then applied to the neuron, which will fire if the membrane potential exceeds its threshold value. If a spike is caused, a 1 is saved, otherwise 0 is saved. The data is then pooled, cutting the length and width of the images in half. This is then input into the second layer, where a similar process occurs. Note the convolution in the second layer has an input channel size of 16 and an output channel size of 64, in order to get 1,024 neurons. The maximum allowed number of neurons is used to maximise the power of the model whilst keeping it at three layers. Finally, the third layer puts the image in its category (type of gesture), and this once again gives a current which may fire a neuron.

In keeping with the currently accepted image classification standards, the cross entropy loss function and Adam optimiser are used.

Evaluation

To evaluate the performance of the convolutional SNN, we compare its accuracy and runtime against a (non spiking) CNN that is identical to our model, apart from replacing the LIF neurons with ReLU

activation functions. Three tests were performed, and the mean accuracy and runtime are displayed. All values to 3 decimal places.

	<u>Convolutional SNN</u>	<u>CNN</u>
Accuracy/% (1)	92.176	73.760
Accuracy/% (2)	93.511	73.378
Accuracy/% (3)	93.922	75.668
Accuracy/% (Mean)	93.203	74.269
Runtime/min (1)	9.482	0.830
Runtime/min (2)	9.601	0.852
Runtime/min (3)	9.518	0.841
Runtime/min (Mean)	9.534	0.841

These results clearly show that the convolutional SNN is significantly more accurate than the CNN, with an average accuracy of 93%, as opposed to 74%. One would expect the CNN to have a much closer accuracy, if not the same, so this suggests that the CNN's parameters were not well optimised. On the other hand, if we were to optimise the CNN's parameters and use these parameters on the SNN, it may have increased the SNN's accuracy even more.

However, the traditional CNN was much faster than the convolutional SNN. The magnitude of the difference in runtime is unexpected but does not necessarily mean that the SNN is computationally more expensive than the CNN. This would be an issue since one of the main advantages of SNNs is that have lower power consumptions.

Overall, the convolutional SNN can predict hand gestures very accurately, in only five epochs, but it takes a long time to do so.

When completing this project, we ran into a few difficulties, most notably the choice of dataset. Initially, we had chosen a different dataset also containing hand gestures, but the model hardly learnt anything about it – it performed about as well as picking a gesture type at random. We were fairly sure that the overall structure of the model was ok, since it achieved high accuracies for some common datasets such as MNIST and Fashion MNIST (allowing for changing the model's parameters), and whilst we expected the accuracy to go down on our own dataset, we were surprised that the model was not learning at all – the loss values were hardly changing at all. In the end, we decided to use another hand gesture dataset, which performed about as well as Fashion MNIST.

How to Run

Required Installations:

- SNN Torch
- PyTorch (installing SNN Torch should automatically install PyTorch)
- Torchvision

Running the scripts (using command prompt):

- To run the main SNN application, enter `python geniusesSubmission.py` in TeamGeniuses directory.
- To run the CNN (for comparison), enter `python cnnTest.py` in TeamGeniuses directory.

References

1. Rooban Sappani, Hand Gesture Recognition, Retrieved from [Hand Gesture Recognition \(kaggle.com\)](#)
2. Lee, Sarwar, Panda, Srinivasan, Roy, Enabling Spike-Based Backpropagation for Training Deep Neural Network Architectures
3. Sengupta, Ye, Wang, Liu, Roy, Going deeper in spiking neural networks: Vgg and residual architectures
4. Eshragian, Ward, Neftci, Wang, Lenz, Dwivedi, Bennamoun, Jeong, Lu, Training Spiking Neural Networks Using Lessons From Deep Learning